

Tasks and Schedulers in Real-Time Systems

1 Tasks, Processes, and Threads

1.1 Processor Context

The CPU of a computer has several state variables. Once they are specified we know the exact state of the CPU. These include:

- Program Counter (location in memory of next instruction)
- Value of each register
- Processor Status Flags
- Stack Pointer

1.2 Tasks or Processes

A *Task* or *Process* is a unit of code and data which is described by one processor context when it is running.

Examples: *The Tasks of Labs 2-5*

A task is usually implemented in C by writing a function.

Types of Tasks:

1.2.1 Periodic Tasks

- Found in Hard-Real time applications
- Examples: 1) Control: Every 10 ms., Read sensors → compute control → output command
2) Multimedia: Every 22.727 μ sec, Get music sample → compute DSP filter → output sample to DAC.
- Characterized by three attributes:
 1. P , Period: the regular time interval between runs of this task.
 2. C , Computing Resources: How much CPU time does it require each time. Obviously $C \leq P$. (C may not be the same each time. Different code branches in the task may take different amounts of time. Use $C = C_{max}$.)
 3. D , Deadline: How quickly must the task be completed after it is started each time tick. $C < D < P$
- Typical Code Structure:

```
name() {
    compute for C seconds;
    return();
}
```

1.2.2 Intermittent Tasks

- Found in all types of applications
- Examples: 1) Send an email every night at 4:00 AM
2) Save all data when power is going down
3) Send a message to plant operator when tank runs low.
4) Calibrate a sensor on startup.

- Characterized by two attributes:
 1. *C*, Computing Resources: How much CPU time does it require each time.
 2. *D*, Deadline: How quickly must the task be completed after it is started. (whenever that happens to be).

- Typical Code Structure:

```
name() {
    compute for C seconds;
    if(! done) return();
    else halt_me();
}
```

1.2.3 Background Tasks

- A soft real time or non real time task.
- Lower Priority
- Will be accomplished only as CPU time is available when no hard real time tasks are ready.
- Characterized by:
 - *C*, Computing Resources: How much CPU time does it require each time between scheduler accesses.

- Typical Code Structure:

```
name() {
    compute for C seconds;
    if(! done) return();
    else halt_me();
}
```

1.2.4 Complex Tasks

- Found in all types of applications
- Examples: 1) Microsoft Word
2) Apache Web Server.
- Characteristics:
 1. Continuous need for CPU time.
 2. Frequent requests for I/O which free up the CPU.
 3. Waits for user input which free up CPU.
- Typical Code Structure:

```
name() {
    while(1) {
        compute for C seconds;
        request_I0(); // also starts scheduler
    }
}
```

I/O activity frees up the CPU because the task must wait, usually a significant amount of time, for the I/O activity on a physical device. It cannot compute between the time it requests the I/O and the time it gets the data or writes the data so now the CPU is free for other tasks.

1.3 Task States

Tasks are in one of four states:

1. **Running**
2. **Ready** to Run (but not running)
3. **Waiting** (for something *other* than the CPU.)
4. **Inactive**

Only one task can be **Running** at a time. A task which is waiting for the CPU is **Ready**. When a task has requested I/O or put itself to sleep, it is **Waiting**. An **Inactive** task is waiting to be allowed into the schedule. It is like Microsoft Word when you are NOT running it.

We'll see later how a task can get into or out of the **Inactive** state.

2 Threads: Smaller Units of Computation

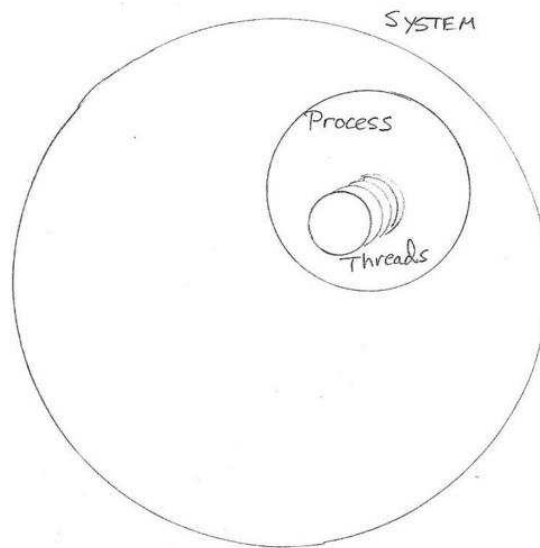
We have talked about:

- Programs
- Tasks
- Subroutines/Functions¹
- Processes
- Threads

Confused yet??

With any complex computing system we naturally break it up into *units* .
For example in the EE472 lab exercise: “tasks” Each unit has

- Code/instructions
- Data
- Context/State
- Resources (memory, I/O devices, semaphores)



¹When I use a slash (/) in a list like this I mean that the two terms are equivalent

A *Program or Process* is a unit of computation with all of the above attributes.

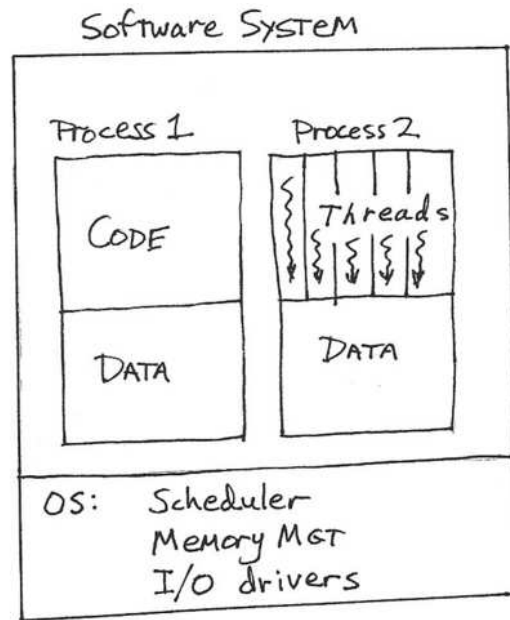
A *Subroutine or Function* is a unit of computation with code, data, and context, but no managed resources.

I am using the term “managed resources” to mean resources which can be assigned and deassigned by the operating system such as memory or I/O devices. Of course a function within a program has its own memory, but if it requests memory from the OS (for example with `malloc()`), the OS assigns it to the program, not to the function within that program.

A *Thread* is a unit of computation with code and context, but no private data. Threads may even share code with each other.

Threads are owned by a program/process.

Threads are like pieces within a program which can be scheduled by the OS independently.



A complete software system with two processes. Process 1 is a “normal” single-threaded process. Process 2 is a multithreaded process.

Scheduler manages 6 units: Process 1 and the 5 threads of process 2.

2.1 Thread Context Switching

Since threads have a smaller context than programs, context switching is faster.

- Only save/restore CPU state
- No need to change memory setup
(with programs in systems bigger than the 16-bit 80188, this can be elaborate).

Why use threads?

An example.

Web server. Web server pseudocode:

```
while(1) {
    OS_read_network();    // get incoming request
    compute;             // figure out what html page to get
    OS_Disk_I/O();        // read disk file for www page
    OS_write_network();   // send page to user
}
```

Alternative 1: Run above as a single process.

Problem: When process is waiting for network disk is idle and vice versa.

Alternative 2: Set up several threads of same code:

Advantage: Scheduler can start a thread which is done with disk I/O while another thread is waiting on network and vice versa.

CPU is used more efficiently and system delivers more pages per second with same hardware performance.

Potential Problem: more time spent context switching.

2.2 Types of threads.

There are lots of possibilities for size of *units*.

The more context a thread contains, the “heavier” it is.

Heavier threads take more time for context switching.

OS designer determines what kind of threads to support — application programmer gets only one option.

3 RTOS Scheduling Concepts

RTOS = Real Time Operating System

3.1 Time

Time is measured by a piece of hardware which records actual clock time: a *real time clock*. We use the terms

“Time Quanta” “Time Slice” “Ticks”

Let’s call the period of time ticks: T .

Typically $T < P_{min}$. Sometimes $T \ll P_{min}$.

Where P_{min} is the shortest period of all tasks in the system. In other words, the operating system measures time in units, T , which are smaller than the shortest period of any task.

3.2 Soft vs. Hard Real Time

Real Time: A software system with specific speed or response time requirements.

Soft Real Time: If the deadlines are not met, performance is considered low.

Hard Real Time: A computer system in which at least one task must meet deadlines in time. If the deadlines are not met, the system has failed.

Super Hard Real Time: Mostly Periodic Tasks: Task periods = the OS time tick, task compute times (C) and deadlines (D) are very short. $P = D = T, C \ll P$.

3.3 Scheduling

At each opportunity, OS asks: If N tasks are **Ready**, which one should I run?

If you can show that scheduler always can achieve deadlines, the system is “deterministically schedulable”.

3.4 Scheduling Goals

1. CPU Utilization:

$$U_{CPU} = 1 - \frac{\text{idle}}{\text{period}}$$

Traditional measure for mainframes. 100% is better.

But $U_{CPU} = 100\%$ is not safe for real-time systems

Goal: low load - 40%, high load - 90%.

2. Throughput: Work units completed per unit time.

3. Turnaround Time: Total time from task start to completion (waiting + execution + I/O)
4. Waiting Time: Choose scheduler which minimizes waiting time.
5. Response Time ** How quickly system responds to external asynchronous events.

Scheduler Types:

3.4.1 Periodic Schedulers

- Infinite Loop
 - Most primitive of all
 - Also known as Non-preemptive Round Robin.
 - `while(1)`

```

{
    task1_fcn();
    task2_fcn();
    task3_fcn();
    ...
}
```
 - Each task **must** voluntarily return to scheduler quickly
 - $C \ll P, P = T$.


```

taskN_fcn()
{
    compute a little bit;
    return();
}
```
- Synchronized Infinite Loop
 - the top of the loop waits on a hardware clock.
 - `while(1)`

```

{
    wait(CLOCK_PULSE);
    task1_fcn();
    task2_fcn();
    task3_fcn();
    ...
}
```
 - Each task **must** voluntarily return to scheduler quickly, as above.
 - $C \ll P, T = P$.

The schedulers above are appropriate when all tasks are periodic and hard real-time. However, because execution times (C) vary, we have to allow extra time in each loop of the non-preemptive round robin schedule (taken up by the “wait()” function).

Diagramming Scheduler Operation

To study scheduling, print out a “Scheduler Diagramming Worksheet” on the the EE472 web site. This chart plots time going down the page and CPU processes (tasks) across the page. We can think of the horizontal axis as the memory space of the computer. We will put a shaded box in a square (or part of a square) when a certain task is running on the CPU. For example, when the scheduler is running, we would shade under the “OS” column. We might also have three tasks labeled A, B, C. We would shade their columns when they are running. Remember, *only one column may be shaded in each row*.

Assume that each line represents $333\mu\text{s}$. Assume that there is a system clock which generates OS interrupts every $1,000\mu\text{s}$. To indicate these, make a mark at the left edge of every third horizontal line.

3.5 Exercises

For Exercises 1 and 2 below, diagram the flow of control between the OS and three tasks:

- The three tasks are labeled A, B, C
- The nature of the three tasks is as follows:
 - A Periodic. $P = .001\text{sec}$, $C = 200\mu\text{sec}$, $D = P$.
 - B Periodic. $P = .002\text{sec}$, $C = 200\mu\text{sec}$, $D = P$.
 - C Periodic. $P = .002\text{sec}$, $C = 200\mu\text{sec}$, $D = P$.

Diagram the tasks A,B,C as described above for the following scheduler policies:

Exercise 1 Non-synchronized, non-preemptive Round Robin

Exercise 2 Synchronized, non-preemptive Round Robin

Exercise 3 Diagram the schedule for the following variation on round-robin:

```
while(1) {
    wait(CLOCK_PULSE);
    taskA();
    taskB();
    taskC();
    wait(CLOCK_PULSE);
    taskA();
}
```

3.5.1 Scheduling Complex and Intermittent Tasks

Scheduler becomes an algorithm or function (instead of a fixed loop).

Order of task execution can no longer be predicted exactly (easily).

Not obvious if schedule can be met.

I/O Sometimes tasks in embedded systems do I/O by directly manipulating hardware. For example, a task may set some bits on a digital output port using `outportb(addr, data)`. These types of operation take very little time so they can be appropriate for periodic hard real time tasks.

On the other hand, more sophisticated devices such as disk drives or network interfaces require complex driver software and importantly take significant time to respond to commands. This type of I/O is typically supported by an embedded operating system which handles all I/O requests through subroutine calls. Since tasks using these devices will have to wait a long time for them to respond, the scheduler is invoked after I/O requests. If a task is waiting for I/O, the scheduler will try to run another task until the I/O is complete.

Exercise 4 Complex task Diagram the schedule for the following tasks:

A $P = 1.0\text{ms}$, $C = 0.233\text{ms}$, $D = 0.5\text{ms}$

B Complex task:

```
while(1){
  compute for 0.233ms;
  I/O Request lasting for 1.333ms;
}
```

Exercise 5: contention Diagram the schedule for the following tasks:

A Periodic. $P = 2\text{msec}$, $C = 233\mu\text{sec}$, $D = P$.

B Periodic. $P = 2\text{msec}$, $C = 656\mu\text{sec}$, $D = P$.

C Periodic. $P = 2\text{msec}$, $C = 233\mu\text{sec}$, $D = P$.

Who starts when 3 are ready?

Exercise 6 Diagram the schedule for the following tasks:

A: Periodic. $P = 2\text{msec}$, $C = 233\mu\text{sec}$, $D = 500\mu\text{sec}$.

B: Complex:

```
while(1){
  compute for 1.5ms;
  Request I/O lasting 0.667ms;
}
```

3.5.2 Foreground/Background

- A mix of hard real-time periodic tasks with Soft real time or non real time intermittent and complex background tasks.
- “Foreground” — the hard real time tasks
- “Background” — the soft/non real time tasks
- Example: Factory Automation Controller:
 - Real time control system (periodic, hard real-time, foreground)
 - Hourly production report generator (intermittent, soft-real time, background)
 - Inventory Database Update (complex, non-realtime, background)

Exercise 7: Foreground/Background Diagram the schedule for the following tasks:

A: Periodic. $P = 2\text{msec}$, $C = 233\mu\text{sec}$, $D = 500\mu\text{sec}$.

B: Complex (background):

```
while(1){
  compute for 0.233ms;
  yield_to_scheduler();
}
```

(`yield_to_scheduler()` is similar to `return()` in our earlier schedulers).

3.6 Delay Functions and Scheduling

Tasks frequently need to wait for a time interval.

Reasons:

- To create time-based events (such as Flash an LED for 0.5 sec).
- To wait for something external to happen such as for a device to be ready for more data.
- To free up CPU time for other processes.
- To determine for themselves how often they run:

```
while(1)
{
    compute;
    delay(20ms); // Run every ~20ms.
}
```

This task sets its own period, P , to 20ms. How accurate?

The worst way to do this is a delay loop such as

```
#define DELAY    22000
for (i=0; i< DELAY ; i++) ; // just loop to use up time
```

Here we assume that we have figured out that 22000 is the right value to use for the time delay we want by trial-and-error or calculation. The worst bad thing about this method is that it doesn't meet the third reason, it uses all the CPU time itself. (There are other problems as well). We can define the call `OS_TimeDelay(int delay)` which does the following:

A better method is to let the scheduler help with delays.

- returns to the scheduler
- sets a software counter to `delay`. (This counter will be decremented every so often by an ISR until `counter == 0`.)
- schedules the next task.
- the task which called `OS_TimeDelay()` will be set to **Waiting** (not started by the scheduler)
- when the `delay` is over, set task to **Ready**

3.7 Dynamic Scheduling

Starting and Stopping Sometimes it is desirable to make a task stop and start. We defined the task state **Inactive**, to describe a stopped task. To implement this, assume that functions are available: `halt_me()` which moves a task from **Running** to **Inactive** states (takes a task out of the scheduler's loop). And `run(task)` which moves a task from **Ready** to **Ready** (puts a task into the scheduler loop).

Exercise 8: Dynamic Schedule Diagram the following tasks:

A Intermittent:

```
while(1) {
    compute for 0.6566 ms;
    yield_to_scheduler();
    after 3 cycles of this halt_me();
}
```

B Complex: run for 1.0ms, then call the OS to do I/O which lasts for 2.0 ms. Repeat.

C Background:

```
while(1) {
    compute for 0.233 ms;
    yield_to_scheduler();
}
```

for

1. Non-synchronized, non-priority-based Round Robin

3.8 Pre-emptive Scheduling

So far, we have required that tasks return to the scheduler frequently so that other tasks get a chance to run in the CPU. This has two problems. First, we have to code our tasks somewhat artificially. They have to break their job up into chunks and keep track. Second, we are vulnerable if some task doesn't "play fair".

The solution to this is a scheduler that can break in to a running task using a regular interrupt such as a clock interrupt. There are many complex issues in having this ability but the value is very great. Preemptive schedulers solve this problem. Whether the task is ready or not, the pre-emptive scheduler can stop it and run something else. Several types of pre-emptive scheduler are described below.

3.8.1 Types of pre-emptive schedulers:

- Pre-emptive Round Robin
 - Regular OS interrupts ("ticks").
 - Scheduler moves to next task each tick.
 - Task()


```
{
            while{1}
            { compute ; }
          }
```
- Priority Based
 - Each task assigned priority by programmer
 - Highest priority task always gets the CPU if ready.
 - (same)
- Rate Monotonic (RMS)
 - As above but priorities are assigned based on period, P .
 - Short period = high priority.
 - Can prove optimality and schedulability (with assumptions).
 - (same)

RMS Schedulability:

e_i = execution time of task i .

P_i = period of task i .

n = number of tasks.

if

$$\sum_{i=0}^{n-1} \frac{e_i}{p_i} \leq n(2^{1/n} - 1)$$

the system **is** schedulable using RMS.

Assumption: All tasks pre-emptable 100% of the time.

- μ Cos-II style Priority Based

- Fixed unique priority and task-specific delay.
- Must tweak delay values, M , to avoid starvation.
- `Task()`

```
{
    compute_some();
    OSTimeDelay(M);
}
```

- Priority Based Round Robin

- All tasks delay 1 tick
- All tasks execute $< \frac{T}{N}$ where T is the OS tick period and N is the number of tasks.
- `Task()`

```
{
    compute_a_little();
    OSTimeDelay(1);
}
```

Preemptive Scheduling Exercises

Exercise 9: Diagram the following tasks:

A Intermittent: Run for 2ms and then `halt_me()`.

B Complex: run for 1.0ms, then call the OS to do I/O which lasts for 2.0 ms. Repeat.

C Background: Run forever.

for these policies:

1. Pre-emptive priority-based: Priority order: 1) B, 2) A, 3) C. (1= highest)
2. Pre-emptive Round Robin (no priorities)

Exercise 10: Assume 5 tasks and for each one $P = 20\text{ms}$, $C=3\text{ms}$, $D=20\text{ms}$. The OS clock is 1ms. How could you set up delay values to implement round-robin scheduling in μ C/OS-II style Priority Based.

3.9 Scheduling Summary

Issue	Scheduling Concepts
How often will each task run?	Task types; Round Robin ; delay functions
Which task runs next?	Task State; Round Robin; Priority Schemes
How do tasks give up CPU?	<code>return()/yield()</code> ; Request I/O ; interrupts/preemption
What if task doesn't need to run all the time?	dynamic scheduling ; <code>halt_me()/start(task)</code>
How can we meet deadlines and make full use of CPU?	Background Tasks

3.10 RTOS Types

Type	C	D	Scheduling
'Super HRT'	$\ll P$	P	Trivial
'Hard RT'	$(1 \sim 5)T$	$(10 \sim 20)T$	Complex
'Soft RT'	$(10 \sim 20)T$	$\sim 100T$	Complex
Non RT	X	X	Trivial

Where P, C, D, T are defined in sections 1.2.1 and 3.1.

3.11 Scheduler Diagramming

Print out multiple copies of the Scheduler Diagramming Worksheet on the next page. We will use them in class to practice analysis of scheduling situations.

We will use each line in the worksheet to represent $333 \mu\text{sec}$. Thus every three lines is one millisecond (our usual operating system time tick). Color in each column when its task should be running.

[illegible]